



Enhancing Network Threat Detection with Random Forest-Based NIDS and Permutation Feature Importance

Mohammed Tarek Abdelaziz¹ · Abdelrahman Radwan¹ ·
Hesham Mamdouh¹ · Adel Saeed Saad¹ · Abdulrahman Salem Abuzaid³ ·
Ahmed Ayman AbdElhakeem⁴ · Salma Zakzouk^{1,2} · Kareem Moussa¹ ·
M. Saeed Darweesh^{1,2}

Received: 14 July 2024 / Revised: 12 September 2024 / Accepted: 23 September 2024 /
Published online: 22 October 2024
© The Author(s) 2024

Abstract

Network Intrusion Detection Systems (NIDS) are critical for protecting computer networks from unauthorized activities. Traditional NIDS rely on rule-based signatures, which can be limiting in detecting emerging threats. This study investigates the effectiveness of the random forest classifier in advancing NIDS capabilities through machine learning. Using the CICIDS-2017 dataset, the data are preprocessed to enhance their quality by removing redundancies. Feature selection and permutation importance were employed to identify the most relevant features. The methodology involves rigorous testing and analysis of the random forest classifier's performance, focusing on f1-score rates compared to other machine learning models. Results demonstrate that by optimizing class weights, applying a custom prediction function and leveraging 26 key features, the random forest classifier achieves an outstanding 99.8% in the weighted f1-score and 93.31% in the macro f1-score in various attack types. This research highlights the potential of machine learning to significantly enhance NIDS effectiveness, offering a robust defense mechanism against evolving cybersecurity threats in modern networks.

Keywords Network intrusion detection system (NIDS) · Feature selection · Permutation importance · CICIDS-2017 · CyberSecurity · Random forest · Rule-based · Signatures

1 Introduction

In today's digital age, Network Intrusion Detection Systems (NIDS) encounter numerous obstacles that hinder their efficacy and efficiency. One major challenge is identifying novel and developing threats, like zero-day attacks and Advanced

Persistent Threats (APTs), that conventional signature-based techniques have difficulty pinpointing. Excessive false alarms can tire security teams, leading to alert exhaustion, whereas failing to detect actual threats can result in missed dangers. The demand for quick detection and response in real-time necessitates NIDS to efficiently handle large volumes of data, sparking worries about scalability and computational burden, particularly in distributed and cloud settings.

Recently, machine learning has become a strong asset for identifying and stopping cyber-attacks. Instead of using fixed rules and signatures, traditional intrusion detection systems can be insufficient in detecting complex or new attacks, while machine learning algorithms can use large data sets to recognize patterns and anomalies signaling a possible attack. These algorithms are able to identify fresh and previously unidentified dangers, giving them a higher level of efficiency compared to conventional systems that rely on signatures. As a result, Network Intrusion Detection Systems (NIDS) have developed to oversee network traffic for any questionable actions and notify security staff of possible dangers [1].

Nevertheless, even though there are benefits, there are also obstacles when applying machine learning and AI to NIDS. This encompasses the need for continuous model training, susceptibility to adversarial attacks, and dependence on high-quality labeled datasets. Factors related to humans, such as the lack of skilled cybersecurity professionals and inadvertent user actions that compromise security, contribute to the complexity as well. It is essential to address these challenges thoroughly to enhance the strength and dependability of current NIDS. Effective security systems are crucial in safeguarding sensitive information stored on networks due to the rising number of cyber-attacks and data breaches [2].

Several different types of machine learning algorithms can be applied to NIDS. For example, supervised learning algorithms can be trained on labeled data to identify known attack patterns. Unsupervised learning algorithms, on the other hand, can detect anomalies in network traffic that indicate a new or previously unknown type of attack. Reinforcement learning algorithms can be used to adjust the system's response to detected threats dynamically. Nevertheless, some challenges are associated with applying machine learning to NIDS [3]. One of the main challenges is the need for large amounts of high-quality training data. In addition, the performance of machine learning algorithms can be affected by the quality and relevance of the features used to represent network traffic.

Despite these challenges, there has been significant research in this area, and many promising results have been obtained [4]. Anomaly-based intrusion detection is one technique for spotting zero-day threats. Nonetheless, there are still challenges and issues in the field, such as enormous data dimensionality [5], effects on computer complexity [6, 7], and computational time [8], despite the development of several anomaly detection techniques. One strategy used by researchers to deal with the data dimensionality issue is the feature importance technique. The feature selection strategy decreases processing time, lessens the impacts of dimensionality, removes features, aids in comprehending data, and enhances the performance of predictive machines [9]. The process of choosing an ideal feature subset that accurately reflects the complete dataset is known as feature selection, which is a component of dimensional reduction [10]. Numerous

studies have been conducted to increase the accuracy of anomaly detection using feature selection strategies, including the works in [11] and [12].

The NSL-KDD dataset, an improved version of the KDD Cup 99 dataset, is used in the majority of the studies. Information Gain, Correlation Based with Naive Bayes and Decision Table Majority Classifier [13], Support Vector Machine (SVM) in [14], and Random Forest in [6] are a few methods that have been suggested and have the potential to increase detection accuracy. However, those techniques were not examined on a big dataset with a lot of attributes. Information gain is used in many research studies on datasets with little attributes available for analysis. The CICIDS-2017 dataset, which has more intricate traits, is employed in this work. The CICIDS-2017 dataset has a lot of characteristics that be analysed for anomaly detection.

This research seeks to overcome these constraints by utilizing machine learning methods, particularly concentrating on detecting anomalies with the CICIDS-2017 dataset. This research aims to improve NIDS capabilities in recognizing and reducing various network intrusions by utilizing a random forest classifier and investigating advanced feature selection techniques. The objective is to enhance detection accuracy and operational efficiency in real-world cybersecurity situations by analyzing and validating data empirically.

Accordingly, this work significantly advances machine learning-based NIDS by conducting a comprehensive analysis of current methodologies, encompassing various feature selection and extraction methods, as well as assessment metrics. Beyond a survey of existing approaches, the work contributes novel insights through:

- Applying and experimenting with a permutation feature selection model.
- Utilizing a random forest as its base, this model provides a practical methodology for optimizing NIDS performance with weighted and macro F1-scores.

The work is divided such that the literature review in Sect. 2 introduces novel machine learning-based methods for identifying and preventing network assaults. In Sect. 3, the suggested model and algorithms are examined in-depth and discussed, focusing on the technical specifics and underlying ideas of the proposed solution. Furthermore, in Sect. 4, experimental findings are presented, demonstrating the performance and efficacy of the machine learning-based technique, not to mention quantitative assessments and comparative analyses are given to show how the model performs better than the state-of-the-art systems already in use. The conclusions and contributions are finally summarised in Sect. 5, highlighting the most important learnings from the work.

2 Related Work

The references mentioned in this section form a comprehensive view of how research in NIDS has evolved to tackle the challenges of detecting and mitigating network intrusions effectively in modern computing environments. All of these

sources work together to enhance and advance NIDS. They show a shift from old-fashioned rule-based methods like association rule mining to advanced machine learning techniques such as neural networks, Naive Bayes, Decision Trees, and random forests. Every reference set focuses on various aspects of NIDS advancement: from improving accuracy and effectiveness in classification using new algorithms (e.g. SSO-WLS and IDS-RS), to refining feature selection techniques (like Information Gain and DDE), and testing these methods on various datasets (such as KDD CUP 1999 and CICIDS-2017). Recent advancements in NIDS have seen various innovative approaches to improve detection accuracy and efficiency. To address the issue of producing decision rules, Chung and Wahid employ the simplified swarm optimization with weighted local search (SSO-WLS) method for data classification and the intelligent dynamic swarm-based rough set (IDS-RS) for feature selection [15]. The paper provides a comprehensive system approach for improving the discovering process in SSO rule mining by weighing three defined constants. The testing findings on the KDD CUP 1999 dataset for the proposed hybrid network intrusion detection system using IDS-RS showed 93.3% accuracy in an average of 20 runs.

The authors of [15] suggest a combination of Simplified Swarm Optimization (SSO) algorithm for rule creation and intelligent dynamic swarm-based rough set (IDS-RS) for selecting features. This mixture is intended to improve the precision and effectiveness of identifying network intrusions. The system employs Single Sign-On (SSO) to enhance the creation of decision rules by modifying three specified constants, while the IDS-RS technique picks the most important attributes from the dataset. The model was tested by the authors on the KDD Cup 1999 dataset, with an average detection accuracy of 93.3% across 20 runs. This study shows how merging swarm intelligence and rough set theory can enhance network intrusion detection by simplifying rule mining and feature selection.

In past studies, network intrusions have been identified using association criteria [16–19]. The drawback of association rule mining is that it tends to produce many rules after applying various performance indicators to the whole rule set gathered, thereby increasing the complexity of the system. Recently, the focus of the researchers has shifted to creating an efficient classifier using machine learning techniques and learning from examples. Neural networks have been extensively used in [20, 21] to find abnormality and abuse tendencies. The difficulty with neural networks is that, when employed with a large dataset like KDD Cup 1999, they require more training time.

Two of the most well-known machine learning methods used in NIDS are Naive Bayes [22] and Decision Trees [23, 24]. The ability to offer high categorization accuracy is Naive Bayes, while limited by its assumption that data features are conditionally independent, can still be a powerful algorithm for specific applications like spam filtering. However, its performance may suffer in more complex scenarios where feature correlations play a significant role. Nguyen and Choi [25] tested the top 10 classifier techniques on network intrusion detection and then proposed a classification. In order to give weight to each feature, authors in [26] propose a feature identification methodology that combines filtered-based and wrapper-based approaches with clustering techniques. The recommended method

is able to identify traits that can improve the accuracy of assault detection. To extract important traits, Chen et al. [27] introduce the tree-seed algorithm (TSA). By eliminating redundant characteristics, the recommended method decreases the dimension of the data, improving the K-Nearest Neighbor (KNN) classifier's performance.

A method known as Discrete Differential Evolution (DDE) approach employing the C4.5 Machine Learning algorithm is covered in [28]. The suggested technique produces 16 relevant characteristics with a classification accuracy of 99.92%. Contrarily, despite the fact that many academics employ Information Gain as a feature selection approach, there is not much research on how to determine the minimal weight or rank score from the Information Gain outcome. This score indicates how much the attributes relate to the classification. The scores that were employed by the researchers in [29] and [30] are, respectively, above 0.4 and above 0.001. In [31], the minimal weight score of 0.8 is taken into account. In contrast, to get the greatest accuracy, researchers in [32] remove characteristics one at a time while using the classifier algorithm.

Kurniabudi et al. [33] conduct a feature analysis of the dataset using Information Gain, a well-liked feature selection technique that assesses the decrease in entropy of the target variable (The class labels) when a certain feature is introduced. The top 10 attributes with the most information gain, according to the authors, include features like packet length, protocol type, and destination port. The performance of several machine learning techniques, such as decision trees, random forests, and KNN, is also evaluated by the authors on the CICIDS-2017 dataset. They discovered that the random forest method has a 99.96% accuracy rate and a 0.09% false positive rate. Using the CICIDS2017 dataset, the work in [34] suggests a hybrid feature selection strategy and machine learning algorithms for identifying computer threats. Using a real-world dataset, the research tries to enhance the performance of machine learning models for intrusion detection. The results of the tests demonstrate that, in comparison to employing all characteristics, the suggested hybrid feature selection technique can enhance the capability of machine learning models for detecting computer threats. The random forest approach, which beats existing machine learning algorithms like decision trees and SVMs, has an accuracy rate of 99.88%, according to the authors.

The suggested methodology in [35] included several steps, and the key objective was to maximize the macro f1-score regarding class detection. These included data gathering, preparation, feature engineering, model evaluation, and tuning. In this study, the best macro f1-score of 0.87 was given by the 26PI-RF model, with the precision being 0.90 and recall 0.86. While, on the other hand, Q-69PI-RNN does well in the anomaly detection of the major class but poorly on the minority class with a macro f1-score of 0.73 and a micro f1-score of 0.78. The study also indicated how efficiently the SMOTE technique handled the problem of class imbalance and showed that in some cases, PI-based dimensionality reduction outperforms PCA with Q-36PI-MLP model with a macro f1-score of 0.22 on CICIDS 2018 dataset. The authors in paper [36] study the application of several machine learning methods- Random Forests, Decision Trees, CNNs, and ANN-on the unbalanced datasets CIC-IDS2017 and CSE-CIC-IDS2018 for network intrusion classification. The best

performing model results belong to the CART decision tree with F1-score from 96.87% to 99.87%. In order to interpret the model outputs, the research also used explainable AI techniques, namely LIME and SHAP. This highlighted the difficulties in producing consistent and understandable explanations, especially in multi-class scenarios. While in study [37] implemented a DDoS detection model based on a Random Forest algorithm on the environment of SDN designed based on Mininet. The researchers identified 20 key data elements that affect the efficiency of DDoS detection and obtained an accuracy of more than 99% and a 99.985% F1 Score.

The paper [38] compares the three strategies of feature selection, including Gini importance, permutation importance, and drop-column significance, which are used to decide upon important parameters like destination port and inter-arrival time using Decision Tree and Random Forest algorithms on the CICIDS2017 dataset by balancing computing cost and accuracy. In the results, it was showed that the selected features concerning intrusion detection features are robust, as the reduction in the number of features from 69 to 10 features yields an F1 score difference of about 0.2%, a false positive rate of almost 0, and an F1 score difference of 0.007 for the false negatives.

The MachineLearningCSV data from the CICIDS-2017 dataset was used in the study [33]. It was converted into an ARFF file so that Weka software could analyze it. Information Gain was used to select features, and then different algorithms, such as Random Forest, Bayes Net, Random Tree, Naive Bayes, and J48, were used for classification. 10-fold cross-validation was used to assess performance metrics like accuracy, precision, and execution time. The findings showed that the J48 classifier performed marginally better with 99.87% accuracy with 52 features, while Random Forest got the greatest accuracy of 99.86% with 22 features. This shows how feature selection may be used to improve anomaly detection performance. However, in paper [39] the authors follow the benchmarking approach and uses the CICIDS2017 dataset in order to evaluate a set of machine learning algorithms with respect to their appropriateness for anomaly-based intrusion detection systems. It includes data pre-processing followed by training and testing using ten well-known supervised algorithms, namely ANN, DT, k-NN, NB, RF, SVM, and CNN, together with unsupervised algorithms such as EM and SOM. The results represent the efficiency of the models in the detection of various kinds of attacks: the ANN model achieves 99.31% accuracy, the RF model achieves 99.54% accuracy, and the 1-NN model achieves 99.49% accuracy. The EM model only demonstrates 60.06% accuracy, and the SOM model achieves 59.06% accuracy.

For this paper [40], the research was centered on the CICIDS2017 dataset for the development of three AID Systems using the machine learning algorithms KNN, improved KNN, and LOF by applying the algorithms to the dataset previously prepared with a high standard by eliminating redundant characteristics. The results showed that LOF performed better among them with an average accuracy of 90.5% and detection rate of 92.74% for zero-day-attack, hence establishing the efficiency and effectiveness of the proposed models in detecting new malware threats. Meanwhile, paper [41] focuses on feature selection and predictive modeling using the R language while analyzing the CIC IDS-2017 dataset with the aid of a number of machine learning techniques. Models such as Artificial Neural Networks and

Random Forest are used to identify network connections as either benign or malicious. The outcomes showed a high prediction accuracy of about 90.909%, demonstrating the dataset's potential to improve Intrusion Detection Systems and successfully addressing class imbalance issues. For the purpose of checking the efficiency of machine learning algorithms in intrusion detection, the proposed work in [42] had a comprehensive experimental framework, which included 12 experiments focused on two defense modules and four kinds of attacks: hulk, TCP flood, slowloris, and slowhttptest. It involved the use of network traffic capture for data gathering, flow records conversion, and deep neural networks, random forests, and decision trees methods for analysis. The models' performance on the USB-IDS-1 dataset significantly declined, especially under varying attack and defense conditions, despite achieving precision, recall, and F1 scores of approximately 0.96, 0.97, and 0.96 on the CICIDS2017 dataset. This highlights the difficulties in transferring models between different environments.

3 Methodology

Overall, Network Intrusion Detection Systems (NIDS) can be classified according to the method they use to detect intrusions: either through misuse detection or anomaly detection. Misuse detection finds attacks by comparing network activity to pre-defined attack patterns [43], while anomaly detection, utilized conceptually in this study, identifies abnormal network behavior. Nevertheless, we opt for a supervised learning method in this research instead of conventional anomaly detection techniques. The Random Forest classifier, which is a supervised algorithm, learns from labeled data to identify both normal and malicious behaviors. While anomaly detection is typically linked to unsupervised learning, in this case it involves recognizing unusual network patterns by utilizing labeled data in a supervised setting [44]. The methodology section in this paper mainly covers the model workflow, data leakage, dataset, data types optimization, data preprocessing, under-sampling methods, RandomizedCV for hyperparameter searching, analyzing feature importance, and using the Random Forest Classifier for best results.

3.1 Model Work flow

The flowchart in Fig. 1 provides an overview of the methodology for developing a machine learning-based Network Intrusion Detection System (NIDS) using the CIC-IDS-2017 dataset. The process consists of the following stages:

1. **Gathering Data:** The CICIDS-2017 dataset, consisting of 2,830,742 entries and 79 features, is employed, encompassing various forms of cyber attacks.
2. **Data Preprocessing:** Dealing with missing or unreliable values, removing duplicates, and dropping highly correlated or redundant features reduces the dataset to 48 features.

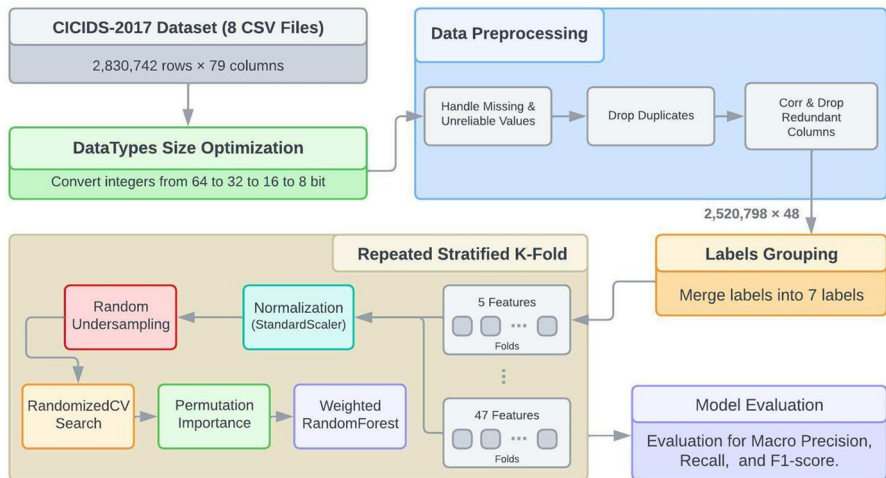


Fig. 1 Overview of model architecture

3. **Optimization of Data Types:** In order to enhance performance, data types for features are transformed into smaller bit forms when possible.
4. **Grouping Labels:** 7 categories are created by merging labels to simplify classification and enhance model generalization.
5. **Cross-validation:** Performing repeated stratified k-fold cross-validation helps to achieve equal representation of all classes in both training and testing datasets.
6. **Selecting Features and Preparing the Model:** The dataset is balanced using random undersampling, normalization is applied, and key features are chosen.
7. **Training the Model:** Hyperparameters are tuned using RandomizedCV search, feature importance is evaluated, and a Weighted Random Forest classifier is trained.
8. **Model Assessment:** The model's overall performance is assessed through Weighted and Macro Precision, Recall, and F1-Score evaluation.

Additional information about every step is explained in the upcoming subsections of the methodology.

3.2 Data Leakage

It is important to mention that to prevent data leakage, the dataset was consistently divided into folds during the entire process. Data leakage happens when the model is built using information not included in the training dataset, resulting in overestimating performance. In order to avoid this, actions like undersampling, normalization, hyperparameter tuning (using RandomizedCV), and feature selection were implemented solely on the training set to guarantee that no data from the test set influenced these processes. Through the use of repeated stratified k-fold cross-validation,

we made sure that each section was kept separate, thus avoiding any unintentional data leakage that could have affected the model's evaluation results.

3.3 Dataset Specifications

The CICIDS-2017 dataset [45] is a freely available dataset that is frequently used in research on NIDS. It was created by the Canadian Institute for Cybersecurity (CIC) at the University of New Brunswick. The dataset contains a rich collection of network traffic, including both benign and malicious activity. It consists of more than 80 network traffic parameters, such as packet size, protocol type, and destination port, that were extracted from raw network traffic that was observed in a real-world network environment [46] The CICIDS 2017 dataset continues to be suitable and efficient in 2024 because it includes a wide range of up-to-date attack types, top-notch labeled data, and a practical network setting that accurately reflects real-life situations. This is reflected by the continuous use of the dataset which can be observed in papers [47] and [48] to measure and compare performance metrics between different models and techniques. In contrast to CICIDS 2018, which is targeted to be used with big data frameworks like Spark making it out of scope for this paper's study, and the previous NSL-KDD dataset, CICIDS 2017 offers a wider variety of attack types and remains pertinent for modern NIDS studies. Moreover, datasets like CICDDoS2019 is more specialized, focusing on DDoS attacks, which might limit the study's general applicability to various attack types. Similarly, CICEV2023 is more focused on vehicle-to-everything (V2X) communication threats, which may not align with this paper's broader goal of improving network intrusion detection across various attack scenarios. Its broad application in the research field establishes a strong standard for evaluating new techniques, guaranteeing the reliability and accuracy of results. The dataset's recognized status, consistent usage in research, and backing from the academic community emphasize its ongoing importance and effectiveness in enhancing NIDS technologies.

The dataset includes nine different attack scenarios, including DoS, DDoS, port scanning, and botnet attacks, in addition to benign traffic from a variety of activities, including email, web browsing, and file transfer. This dataset is commonly used in studies to evaluate the effectiveness of different approaches and determine how effectively NIDS algorithms work. It provides an exhaustive and practical methodology for NIDS assessment, and several network security research projects have used it. The dataset was created by capturing network traffic from a real-world network environment, which makes it a more realistic picture of actual network traffic than synthetic datasets. The focus of this research is to take advantage of CICIDS-2017's proven structure and effectiveness in simulating real-world network conditions despite the emergence of new attacks and challenges in recent datasets. Since there is a strong emphasis on machine learning methods, the dataset offers a thorough and useful foundation for evaluating NIDS, in line with the technologies and restrictions outlined in the research. The dataset has a total number of 79 features which demonstrates the weights of each feature before clustering. For more details, Table 1 [45] clarifies the dataset details.

The CICIDS-2017 dataset includes a wide variety of features necessary for comprehending network traffic behaviors. Flow Duration, Total Forward Packets, and Total Backward Packets are flow-based characteristics which measure the amount of data exchanged between the sender and receiver. These characteristics, in addition to Flow Bytes/s, offer a comprehensive view of the amount of data flow, which is crucial for monitoring network traffic health. Features related to time, like Flow IAT (Inter-Arrival Time) and its variations in both directions, provide information about the timing differences among packet transfers. These characteristics are important for detecting abnormal delays or sudden increases in network activity, which can signal malicious actions.

Features based on packet length, such as Average, Minimum, and Maximum Forward Packet Length, as well as Backward Packet Length, indicate the packet sizes in both directions. These characteristics aid in identifying irregularities in packet size distribution, which may indicate possible attacks. Also, characteristics like the number of SYN Flags, ACK Flags, and PSH Flags in the header help in capturing distinct behaviors of TCP flags, commonly employed in detecting particular attack forms like SYN floods. Header Length characteristics also offer information on the protocol overhead and are beneficial for identifying potential tampering in packet headers.

Lastly, features based on rates such as Packets/s and Bytes/s quantify the speed at which data and packets are transmitted across the network. Abrupt increases or decreases in these rates may indicate an attack, like a denial-of-service (DoS) attack. Furthermore, characteristics concerning Window and Segment Size, like Initial Window Bytes and Minimum Segment Size, aid in examining flow control and segmentation in TCP connections. These characteristics are essential for recognizing assaults that misuse TCP's flow control mechanisms. In total, the extensive range of

Table 1 CICIDS-2017 dataset [45]

	Category	Total
BENIGN	BENIGN	2,273,097
DOS	DDoS	128.027
	DoS slowloris	5796
	DoS Slowhttptest	5499
	DoS Hulk	230,124
	DoS GoldenEye	10,293
	Heartbleed	11
	PortScan	158,930
Bot	Bot	1966
Brute-force	FTP-Patator	7938
	SSH-Patator	5897
Web attack	Bruteforce	1507
	XSS	652
	SQL Injection	21
Infiltration	Infiltration	36
Total		2,830,743

features in CICIDS-2017 offers valuable information on network traffic, making it a valuable dataset for training intrusion detection machine learning models.

A training set and a testing set are created from the dataset. About 80% of the features are in the training set, while the remaining 20% are in the testing set [49]. Through this divide, researchers are able to train their models on a sizable collection of labeled cases and assess their effectiveness on a different collection of instances that were not utilized for training. It contains data from several network protocols, including TCP, UDP, and ICMP. This variety enables researchers to assess the performance of their NIDS methodologies across various forms of network traffic.

3.4 Data Type Optimization

At the beginning of the data preprocessing, the data has a large size, surpassing 1.7 + GB, which consumes a lot of computation. In order to tackle this problem and enhance effectiveness, the data's data types were decreased. More precisely, integer data types were decreased from 64-bit to 32-bit, 16-bit, and 8-bit, based on the value range, resulting in a model size of 848.6 MB after datatype optimization. This decrease in memory usage is important for managing large datasets or for deploying models on devices with limited resources. Moreover, reducing data type sizes improved the model's performance by cutting down on the number of operations needed for each computation. The size of floating-point (float) data types was maintained to prevent accuracy loss or rounding errors, which could impact the model's precision.

3.5 Data Pre-processing

Data pre-processing seeks to prepare the raw data for analysis by addressing issues, including missing values, noisy data, and inconsistent formatting. Data pre-processing involves a number of different procedures [50]. Prior to taking any action, it is critical to identify any missing or insufficient data points in the collection. This includes completely removing instances of missing data or replacing the missing values.

To guarantee that all characteristics are on the same scale and increase the model's accuracy, the data must be normalized. This might entail normalizing the data to a certain range or standardizing the data to have a mean of zero and a standard deviation of one [51]. To make sure the model is trained on a representative sample of the data and that it can generalize successfully to new, unseen data, it is crucial to divide the data into training and testing sets. This can include applying the stratified sampling and k-fold cross-validation strategies.

The feature selection process is a crucial component of data pre-processing. To do this, choose the dataset's most pertinent characteristics to add to the model. As it minimizes the dimension of the data and gets rid of unnecessary or redundant characteristics, feature selection can help the model be more accurate and efficient [52].

In datasets for NIDS, class imbalance is a problem that must be addressed as part of the pre-processing of the data. An unbalanced class is one where one class (like

benign) is substantially more prevalent in the dataset than another class (like intrusive). As a result, when used on the minority class, the model can be biased and underperform. This work uses strategies like undersampling the majority class to get around this problem.

One of the techniques used is the Repeated Stratified KFold cross-validation approach, which is a popular machine learning approach for assessing how well models perform on a dataset. Using this technique, the dataset is divided into k folds, each of which must have a proportionate representation of the target variable [53]. When the target variable is unbalanced, it is crucial that each fold of the stratified method has a balanced representation of the target variable. By iterating through the folds, the KFold technique uses one fold for testing and the rest of the folds for training the model. This process is performed several times to guarantee that the model is assessed using various subsets of the data. With the repeated method, the KFold procedure is repeated several times with various random seeds to lessen the effect of data unpredictability and guarantee the model's performance is reliable.

3.5.1 Missing Values and Unreliable Values

One of the most important steps in the pre-processing of data is the removal of invalid or missing data points from the dataset. Missing data might lead to biased or incorrect models if it they are not handled properly. Missing data can occur for several causes, including errors in data collection or data entry [54]. The missing values can be imputed using methods like mean imputation, median imputation, KNN imputation, or even instances with missing values can be removed. Instances with incorrect values that could harm the model's performance have been removed. A total number of 1358 nan values and 1509 infinite values have been found and dropped in addition to the ones that contain infinite values.

3.5.2 Correlation and Dropping Redundant features

An important stage in data pre-processing is eliminating features from the dataset that are duplicated or closely linked. High levels of correlation between features are referred to as highly correlated features, whereas redundant features provide the same information as other features in the dataset. When a model has these traits, it overfits, leading the model to identify noise in the data rather than underlying patterns. To address this issue, correlation analysis was done in order to identify strongly linked traits and delete one of them [55]. To reduce the dimensionality of the dataset and improve the performance of the model, the redundant feature was removed when two features were substantially related to one another and one of them was redundant.

The process began by calculating the correlation values, which is the Pearson's correlation, between the different features of the dataset. This is done by taking the absolute value of all the features with correlation values of 0.8 or higher and displaying them in the matrix. Thus, 8 features were identified as redundant values due to having a zero std; which are `bwd_urg_flags`, `fwd_avg_bytes/bulk`, `fwd_avg_packets/bulk`, `fwd_avg_bulk_rate`, `bwd_avg_bytes/bulk`, `bwd_avg_packets/bulk`,

bwd_avg_bulk_rate. In addition, 23 features which are shown in Table 2 were identified as correlated values because they have more than or equal to an absolute 0.95 score, so they were dropped from the dataset. Figure 2 shows the total amount of features left after removing both redundant and correlated values.

3.5.3 Data Duplicates

In the dataset, 307,078 duplicate values have been discovered. This discovery emphasizes the value of locating duplicated data within the data pre-processing step and taking appropriate action. Duplicate data can be caused by a variety of things, such as errors in data collection, data entry, or data processing. If duplicate data are not handled properly, it results in biased or inaccurate models. To address this issue and avoid bias in the machine learning model, it is essential to remove all but one of the repeated instances from the dataset.

3.6 Data Grouping and Data Sampling

Comparable occurrences are grouped based on specified criteria, such as the type of attack or other relevant characteristics, rather than the IP address of origin, as IP addresses are not provided in the CICIDS2017 dataset. This form of data grouping helps to identify patterns and connections within the data, revealing attacker strategies and network vulnerabilities without relying on static identifiers like IP addresses. Grouping the data also reduces the dataset's imbalance, facilitating more efficient model training.

On the other hand, data sampling includes choosing a portion of the data for machine learning research [56]. Sampling the data can assist in addressing problems like class imbalance or data skewness as well as assisting in reducing the computational complexity of machine learning methods. A variety of sampling methods, including stratified sampling, undersampling, and random sampling,

Table 2 Selected features for the NIDS model

Feature No.	Feature name	Feature No.	Feature name
1	fwd_iat_total	13	avg_bwd_segment_size
2	total_backward_packets	14	fwd_packets/s
3	total_length_of_bwd_packets	15	fwd_iat_max
4	subflow_fwd_packets	16	idle_mean
5	subflow_bwd_packets	17	idle_max
6	subflow_bwd_bytes	18	idle_min
7	subflow_fwd_bytes	19	syn_flag_count
8	fwd_packet_length_std	20	cwe_flag_count
9	avg_fwd_segment_size	21	fwd_header_length
10	bwd_packet_length_mean	22	average_packet_size
11	bwd_packet_length_std	23	ece_flag_count
12	max_packet_length		



Fig. 2 Correlation matrix of remaining 48 features

are utilized [57]. In addition, Table 1 [45] shows the dataset before classifying it into related groups, and Table 3 displays the percentages that each sort of assault deducts from the total dataset after grouping. It was found that various classes, including PortScan, Brute-Force, Web Attack, Bot, and Infiltration, were minority classes, while the classes BENIGN and DoS were majority classes after classifying the data in the system using the CICIDS-2017 dataset.

This outcome highlights the issue of the dataset's class imbalance, in which certain classes are obviously more prevalent than others. Due to its propensity to favor the dominant class and potential difficulty identifying the minority classes, class imbalance has a negative effect on the model's performance [58].

3.7 Random Under-Sampling

After preprocessing the dataset, the numbers for DoS and BENIGN were very different from those for the other attacks; as a result, undersampling has been done to balance the distribution of the classes. Since oversampling techniques,

Table 3 Dataset post data-grouping

Category	Category grouped	Total	Percentage
BENIGN	BENIGN	2095057	0.831109
DDoS	DOS	321770	0.127646
DoS slowloris			
DoS Slowhttptest			
DoS Hulk			
DoS GoldenEye			
Heartbleed			
Portscan	Portscan	90694	0.000014
FTP-Patator	Bruteforce	9150	0.003630
SSH-Patator			
Web attack bruteforce	Web attack	2143	0.000850
Web attack-XSS			
Web attack-SQL injection			
Bot	Bot	1948	0.000773
Infiltration	Infiltration	36	0.000014

especially ones that generate synthetic samples, would increase the computational cost of the model, as a result, by undersampling the aim is to not overwhelm the model with excessive samples for the majority class which may introduce noise and negatively impact model performance. The sum of minority classes—PortScan, Brute-Force, Web Attack, Bot, and Infiltration—was specifically multiplied by $(0.8/0.2)$, giving the number of samples for the majority classes to be sampled. Therefore, in this work, the majority classes were randomly under-sampled to present 80% of the original dataset, while the minority classes represented 20% of the data. Furthermore, The samples of the majority classes are distributed between DoS and BENIGN according to their proportions.

When the minority classes are underrepresented in the dataset, this method of undersampling can be helpful since it can help to balance the classes [59]. The algorithm can more precisely detect and categorize instances of these sorts of assaults by making sure that the minority groups are fairly represented in the sample. The algorithm still captures the patterns and traits of the majority classes while avoiding any potential bias that could result from an unbalanced dataset by distributing the remaining percentages across these groups. The system is capable of identifying and classifying instances of both minority and majority classes by carefully choosing the percentages allotted to each class while also eliminating any potential biases that result from an unbalanced dataset.

By limiting the quantity of data available by under-sampling, the model is compelled to concentrate on the most important traits that are essential for identifying occurrences of attack. Overfitting be avoided by keeping the model simple and preventing it from detecting irrational patterns or correlations in the data. Furthermore, under-sampling can help cut down on computational requirements and training time

because the model is trained on a smaller dataset. This is especially useful when the training process requires a lot of time and resources due to the size of the dataset. The data after undersampling are shown in Table 4. It is clear that the dataset balance has improved, reducing the impact of an unbalanced dataset.

3.8 Hyperparameter Fine-Tuning

Fine-tuning hyperparameters is essential for improving the effectiveness of machine learning models. RandomizedCV provides a more practical option over conventional GridSearch. GridSearch methodically considers every potential combination of predefined hyperparameter values, while RandomizedCV randomly selects a set number of hyperparameter combinations from a specified range. This probabilistic strategy frequently produces superior outcomes in a shorter timeframe, particularly when managing numerous hyperparameters or wide parameter intervals. For this study, RandomizedCV will be utilized with 5 stratified k-fold cross-validation folds and 2 iterations, in order to guarantee rigorous model assessment and prevent overfitting. The F1 macro score, explained in the results section, offers a holistic measure of performance by averaging F1 scores for various classes, providing a fair assessment of model accuracy even when there is class imbalance. The use of RandomizedCV along with the F1 macro score guarantees a more effective and informative assessment of model performance. Table 5 provides a summary of the hyperparameters used to train each model.

3.9 Feature Importance

The level to which each feature or variable in the dataset contributes to the precision and effectiveness of the model is referred to as feature importance [60]. The permutation importance method, which includes randomly rearranging the values of each feature and calculating the effect on the model's performance, is a popular method for gauging feature relevance. Features are ranked in importance according to how much of an influence they have on the model's performance. However, measuring feature importance can be difficult when working with correlated features.

In these situations, permutation importance is not a useful tool for determining the relative importance of features since it does not completely account for the

Table 4 Data after undergoing under-sampling

Attack Type	No. of samples	Percentage
BENIGN	360514	0.693491
DoS	55369	0.106509
Portscan	90694	0.174461
Bruteforce	9150	0.017601
Web attack	2143	0.004122
Bot	1948	0.003747
Infiltration	36	0.000069

Table 5 Hyperparameters used for each model

Model	Hyperparameters
RandomizedSearchCV	estimator=RandomForestClassifier(class_weight="balanced") cv=RepeatedStratifiedKFold(n_splits=5, n_repeats=2) n_iter=3 scoring="f1_macro"
permutation_importance	estimator=RandomForestClassifier(class_weight="balanced", **best_randomizedcv_params) n_repeats=3 scoring="f1_macro"
RandomForestClassifier	class_weight="balanced" **best_randomizedcv_params

impact of each associated characteristic [61]. In addition, removing the highly correlated features helped assist in addressing this problem; using this technique, strongly correlated characteristics are clustered together, and from each cluster, one feature is selected to be a component of the model. Removing redundant or interchangeable parts enables the model to more accurately reflect the underlying connections and patterns in the data.

Furthermore, feature importance provides a comprehensive collection of network traffic attributes that allow researchers to assess different NIDS implementation methods. The information also includes labels that categorizes each instance of network traffic as either benign or malicious and identify the specific sort of attack for the malicious traffic. Additionally, CICIDS-2017 is appropriate for training and testing complicated machine learning models due to its original size, which is relatively huge (about 2.8 million instances).

The average importance of each feature to the Random Forest classifier's performance across all cross-validation folds is shown in Fig. 3. The destination_port stands out as the most important feature, with the highest permutation importance score, showing its crucial role in differentiating normal and anomalous network behaviors. This is then succeeded by fwd_iat_min and init_win_bytes_forward, which also show significant relevance, albeit to a lesser extent than the destination port. These characteristics illustrate important elements of network traffic patterns, such as the gaps between arrivals and the size of data transmission windows.

Additional aspects like bwd_packet_length_max and fwd_urg_flags show little impact, evidenced by their considerably lower permutation importance ratings. These results indicate that even though the dataset has numerous features, only a small number significantly affect the model's performance. The features that are still available have little impact on the model's ability to make predictions and could add unnecessary noise or duplication if not handled with caution.

The significance lies in emphasizing the most crucial network traffic attributes, like port data and packet timing, to enhance classification accuracy and possibly simplify models by removing unnecessary features. This group of characteristics serves as the foundation for enhancing the NIDS in order to attain both high precision and computational effectiveness.

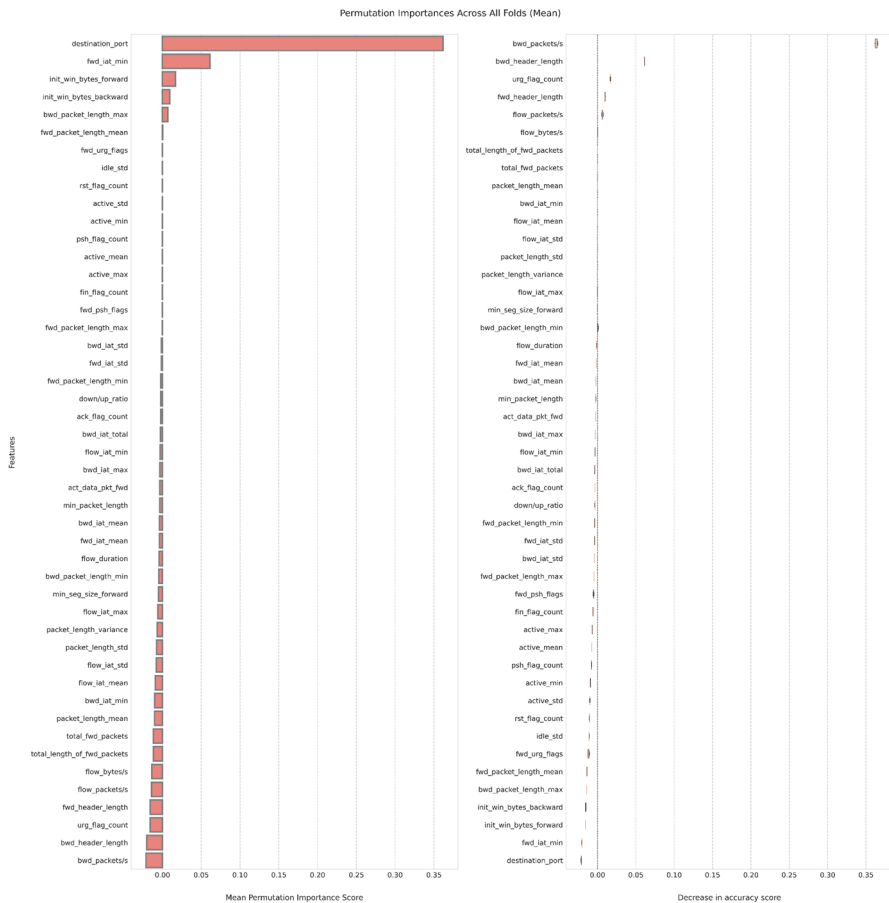


Fig. 3 Permutation score of the 48 features

The issue of impurity-based feature significance for trees being substantially biased and favoring high cardinality features is addressed with the use of the feature importance approach known as permutation importance [62]. High cardinality features have a higher likelihood of being chosen to be important by impurity-based feature importance for trees, such as Gini importance or mean decrease impurity. This is because they can induce more splits. Contrarily, permutation importance evaluates a feature’s significance by randomly permuting its values and assessing the impact on the effectiveness of the model. This method can give a more accurate estimate of feature relevance and does not favor high cardinality features.

Additionally, by offering a more thorough view of feature importance, permutation importance can assist in addressing the problem of strong bias towards specific features. Permutation importance takes into account each feature’s contribution to overall model performance, independent of its unique qualities, in contrast to impurity-based feature significance approaches that favor some

features over others [63]. As a result, the importance of permutation can contribute to a more precise evaluation of feature relevance. Measurement of feature importance was a method used to guarantee the validity of the results.

Various models were created for each feature in the dataset, and the values of each model were then randomly generated. This makes it possible to measure the impact of each feature on the model's performance independently. Using this method, the work created 70 different models, one for each attribute in the dataset. Following the randomization of the values of the associated characteristics in the data for each model, the accuracy of each model was assessed. Each characteristic's value was computed as the accuracy gap between the original and the randomized models, with larger gaps suggesting more significance. It is important to note that in this work, the process of developing different models for each feature and randomly generating the values of that feature was repeated four times in order to further guarantee the validity and reliability of the results.

Multiple iterations of the procedure can help lessen the impact of random variations in the data and guarantee consistency and reproducibility of the results. This work was able to confirm the consistency and dependability of the feature significance rankings by carrying out the procedure four times, which proved to be the best number of iterations that compromise between time consumption and satisfactory results. This strategy can assist in reducing the impact of random fluctuations in the data and guaranteeing that the outcomes are not only dependent on chance.

3.10 Weighted Random Forest Classifier

A common classification method with many benefits is the Random Forest Classifier [33, 34]. It is renowned for its robustness in contrast to other classification algorithms, as it is less prone to overfitting. This is because it averages out the predictions from several decision trees, which lessens the influence of any particular trees that could be overfitting the data. Secondly, random forest is versatile and can accept a variety of input data types, including categorical and numerical data as well as missing values. Thirdly, because outliers tend to be separated inside their own trees, a random forest is good at recognizing them.

Algorithm 1 Predict probability class

```

1: Input: Array arr of probabilities
2: Output: Predicted class index
3: Indices Mapping: {0: BENIGN, 1: Bot, 6: Web Attack}
4: max_index  $\leftarrow$   $\text{argmax}(\text{arr})$ 
5: if max_index = 1 or max_index = 6 then
6:   if (arr[max_index] - arr[0]) < 0.6 then
7:     Return 0
8:   end if
9: end if
10: Return max_index

```

The provided hyperparameters have a major influence on the algorithm's performance. This work found that a particular set of hyperparameters worked well for the particular issue at hand. Algorithm 1 describes a custom prediction function where the model is initially trained with the best hyperparameters found in Randomized-SearchCV on the chosen features. In the prediction phase, the standard weighted probabilities are calculated for every class. Instead of simply assigning the class with the highest probability, the algorithm uses a more sophisticated decision-making process. Initially, it detects the class with the highest predicted probability by utilizing the `argmax` function. If the predicted class is Bot or Web Attack (indices 1 or 6), and the difference between its probability and the BENIGN class is less than 0.6, the prediction is changed to BENIGN (index 0). This modification reduces errors where the model could mix up harmless traffic with specific attack types, enhancing the overall accuracy of the classification, especially for less common categories such as Bot and Web Attacks. The `class_weight` option was also set to "balanced" it was set to balanced to help ensure the recall values in the minority class are given more weight, which resulted in classes being weighted inversely in relation to their frequency. These hyperparameters were discovered to deliver the greatest outcomes for the particular issue at hand.

4 Results and Discussion

The Repeated Stratified KFold cross-validation approach was used to address the problem of unbalanced data in the CICIDS-2017 dataset. By making sure that the distribution of classes is preserved during each fold of the cross-validation process, this strategy has been found to be useful in managing unbalanced data. The authors utilized 5-fold cross-validation with 2 iterations specifically, yielding a total of 10 distinct data splits. As a result, they were able to predict the performance of their models on hypothetical data with more accuracy. Previous work [33] that showed how well Repeated Stratified KFold handled unbalanced data for anomaly detection tasks provided the impetus for its implementation. The authors of [33] also used Repeated Stratified KFold to assess how well their models performed on the CICIDS-2017 dataset. They discovered that this method worked well for addressing the dataset's imbalance and gave them a more accurate indication of how well their models performed. Because it has been demonstrated to be efficient in handling imbalanced data and can offer a more accurate estimate of model performance, the Repeated Stratified KFold was utilized in the proposed work. Using this method, the proposed work authors' were able to accurately assess how well their models performed on the unbalanced CICIDS-2017 dataset. To analyze the performance of the model using the random forest classifier, four measurement metrics are used, they are Precision, Accuracy, Recall, and F1-score. Their equations are as follows [64]:

$$\text{Accuracy} = \frac{T_p + T_n}{T_p + T_n + F_p + F_n} \quad (1)$$

$$\text{Precision} = \frac{Tp}{Tp + Fp} \quad (2)$$

$$\text{Recall} = \frac{Tp}{Tp + Fn} \quad (3)$$

$$F1 = \frac{2 \times (\text{Precision} \times \text{recall})}{\text{Precision} + \text{recall}} \quad (4)$$

Precision measures the ratio of correctly identified attacks to all instances predicted as attacks [64]. It shows how accurate a positive prediction (an attack) is made. Accuracy is a metric that assesses the accuracy of predictions, including both true positives and true negatives, in relation to all predictions, such as false positives and false negatives. Recall, also referred to as sensitivity, evaluates the ratio of correctly identified attacks to all instances that are truly attacks [64]. It indicates the model's ability to correctly detect all positive instances (attacks). The F1-score, which is the balanced measure of a model's overall performance in detecting attacks, is calculated as the harmonic mean of precision and recall. This metric aims to minimize both false positives and false negatives in identifying attacks [64].

Precision is a measure of the proportion of true positives among the predicted positives [64]. It provides an indication of how often a positive prediction is correct. Accuracy measures the overall correctness of the predictions, i.e., the proportion of correct predictions among all the predictions [64]. Recall is a measure of the proportion of true positives among the actual positives [64]. It provides an indication of how well the model is able to identify all the positive instances. F1-score is the harmonic mean of precision and recall and provides a balanced measure of the model's overall performance [64].

These indicators were selected due to their ability to offer an in-depth assessment of the model's performance. When evaluating the performance of models in datasets with imbalanced class distributions, precision and recall are especially helpful metrics. A frequently used statistic that offers a general evaluation of the model's performance is accuracy. The F1-score is a valuable indicator for balancing recall and accuracy and showing how well the model performed overall.

To generate satisfactory results, the model ran multiple times with different amounts of features in each iteration. The impact of different feature sets on the model's performance, both with and without the custom prediction function, was evaluated using 5, 15, 26, 36, and 47 features. The feature values were chosen using the `linspace(5, 47, 5)` function, which generates 5 evenly spaced values between 5 and 47. This approach ensures a balanced and systematic selection of feature subsets across the specified range for comparative evaluation. Notable variations in precision, recall, and F1-scores were observed across multiple attack types.

Datasets such as CICIDS-2017 for network intrusion often show a large imbalance in the amount of benign and attack traffic. One example is when the majority of the dataset consists of harmless traffic (normal network activities), with only a small

portion being made up of attack traffic like Bot, Web Attack, or Infiltration [45]. In certain situations, precision may give a false impression. A model may attain a high level of accuracy by primarily predicting the majority class (benign), while demonstrating lower performance on minority attack classes. This may lead to a model that appears to be accurate but is not effective in identifying rare yet important attacks.

The confusion matrix that presented in Fig. 14 demonstrates the effectiveness of the custom prediction function in reducing misclassifications. Initially, the model occasionally misclassified benign traffic as either Web Attacks or Bot Attacks, which could lead to false positives and decreased overall model performance. However, with the introduction of the custom prediction function, the confusion between benign traffic and these specific attack types was significantly reduced. This improvement reflects a more accurate differentiation between legitimate and malicious activities, enhancing the reliability of the model in detecting real threats while minimizing the risk of false alarms. The confusion matrix visually supports this by showcasing the decrease in erroneous classifications, particularly between the benign class and the Web Attack or Bot Attack categories.

Macro F1-score treats all classes equally by calculating the F1-score (the harmonic mean of precision and recall) for each class and then averaging them. This metric ensures that the model's performance on minority classes (like Bot or Infiltration) is given equal weight as the majority class. It highlights whether the model is performing well across all classes, regardless of their frequency, making it ideal for comparing with other studies that might also deal with minority attacks [65]. On the other hand, Weighted F1-score accounts for class imbalance by assigning more weight to the performance on the majority class but still incorporating the performance on minority classes. It provides a balanced view of the model's performance by adjusting the influence of each class based on its frequency in the dataset. This is useful when comparing models, as it highlights overall performance while still being sensitive to the performance on less frequent but important attack types [66].

By using the macro F1-score and weighted F1-score, the performance of the proposed model can be more fairly and comprehensively compared with other studies that also deal with imbalanced data. While some studies may report high accuracy, this could mask poor performance on minority attack classes. By reporting F1-scores, particularly the macro F1-score, a demonstrate of how well the model performs across all attack types is shown, including those that are less frequent but critical. Moreover, using the weighted F1-score allows for a fair comparison with studies that use similar datasets but might have slightly different class distributions.

With 5 features demonstrated in Fig. 4, high precision and recall were achieved for major classes such as BENIGN, Brute-Force, DoS, and PortScan. However, lower recall was noted for the Bot, Infiltration, and Web Attack classes. The introduction of the custom prediction function shown in Fig. 5 resulted in slight improvements, particularly in the recall for Bot and Web Attack classes, leading to an increase in macro mean precision and F1-scores. This highlighted the need for additional tuning to improve detection in minority classes, even though the model performed well for more common attacks.

With 15 features presented in Fig. 6, excellent precision and recall were recorded for major attacks such as BENIGN, Brute-Force, DoS, and PortScan, ranging from

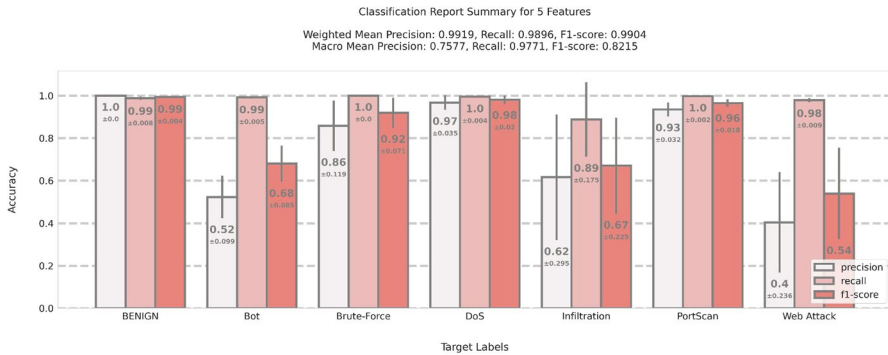


Fig. 4 Simulated attack types using 5 features

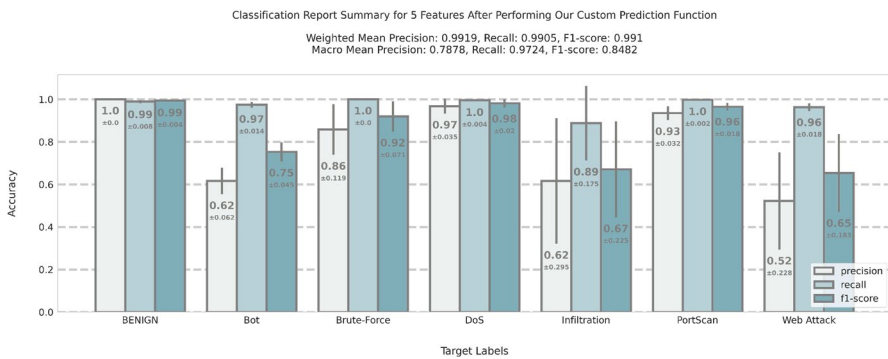


Fig. 5 Different types of attacks using 5 features with the custom prediction function

93% to 100% though a significant drop in recall and F1-scores was observed for Bot and Web Attack classes. In Fig. 7, when the custom prediction function was applied, noticeable improvements were made, especially in the recall for the Bot class and the F1-score for Web Attack. These enhancements resulted in higher macro mean scores, emphasizing the importance of tailored prediction functions for handling imbalanced classes.

Using 26 features yielded the most optimal results as shown in Fig. 8. Without the custom prediction function, the model showed consistently high precision and recall for major attack types, but the Bot and Infiltration scores remained relatively low. The use of the custom prediction function significantly improved scores for Bot and Web Attack, resulting in the highest overall performance displayed in Fig. 9. An enhanced ability to generalize across different classes was noted, as reflected in elevated macro mean scores. It was likely that this feature set balanced complexity and overfitting, effectively capturing essential patterns for detecting various attack types.

In Fig. 10, for 36 features; high scores were maintained for most attack types, although the Bot and Infiltration classes still lagged. Modest improvements were

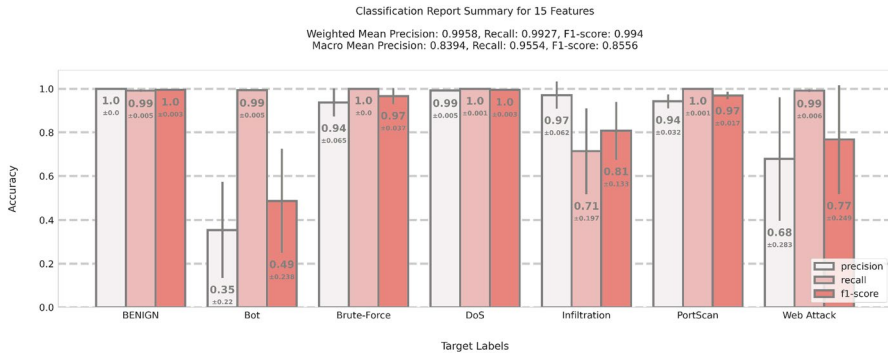


Fig. 6 Different types of attacks using 15 features

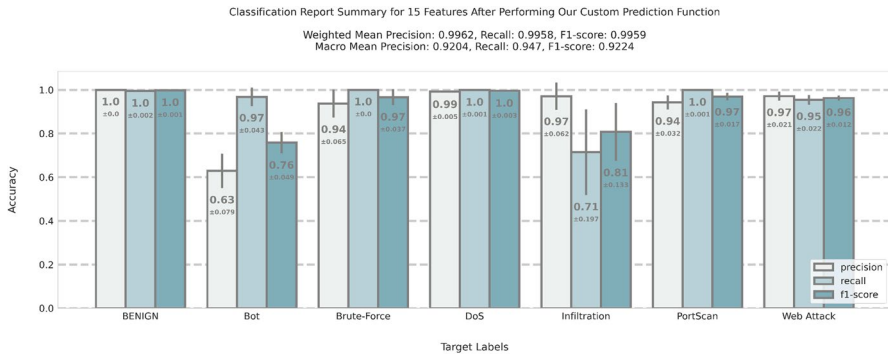


Fig. 7 Different types of attacks using 15 features after using the custom prediction function

observed in these classes when the custom prediction function was applied shown in Fig. 11, and macro scores remained robust. However, performance did not exceed that of the 26-feature model, suggesting that adding more features may not always lead to better results.

With 47 features shown in Fig. 12, the model maintained high precision and recall for most classes, although underperformance in the Bot class persisted. The custom prediction function once again led to improvements in Bot class scores as demonstrated in Fig. 13, though the gains were less significant compared to smaller feature sets. Macro scores remained consistent across the feature sets, reinforcing the notion that increasing the number of features beyond a certain point does not result in substantial performance improvements for minority classes.

The 26-feature model, in combination with the custom prediction function, shown in Fig. 9 achieved the most balanced and superior performance. The optimal balance between feature richness and overfitting was struck, allowing key patterns to be captured without unnecessary complexity. The hyperparameters were well-tuned to address the dataset's class imbalances, resulting in significant improvements in

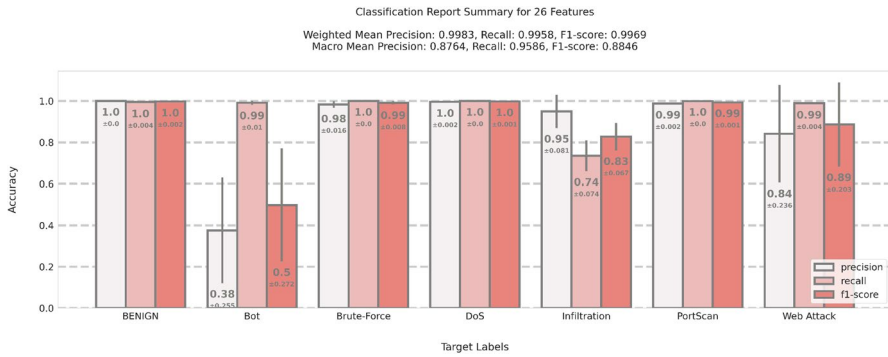


Fig. 8 Different types of attacks using 26 features

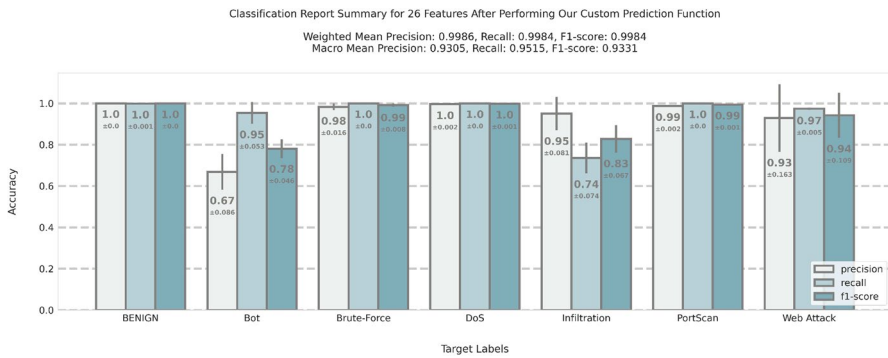


Fig. 9 Demonstration of various attack types and their scores for 26 features with the usage of the custom prediction function

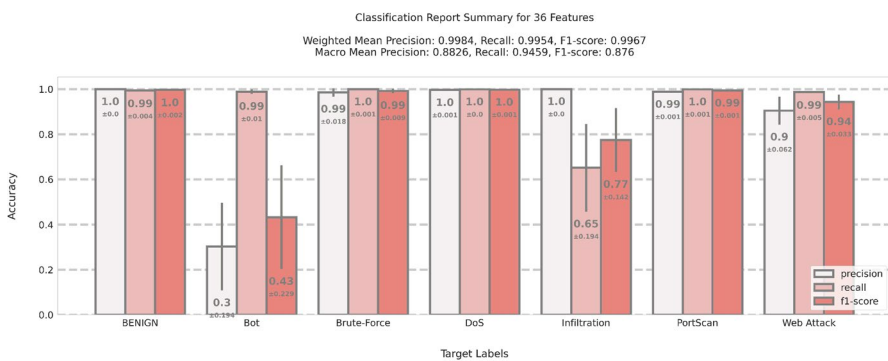


Fig. 10 Scores of different attack types for 36 features

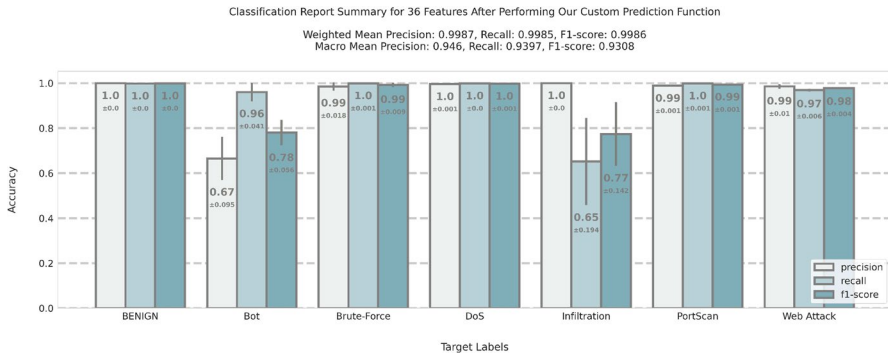


Fig. 11 Attack types metrics for 36 features while using the custom prediction function

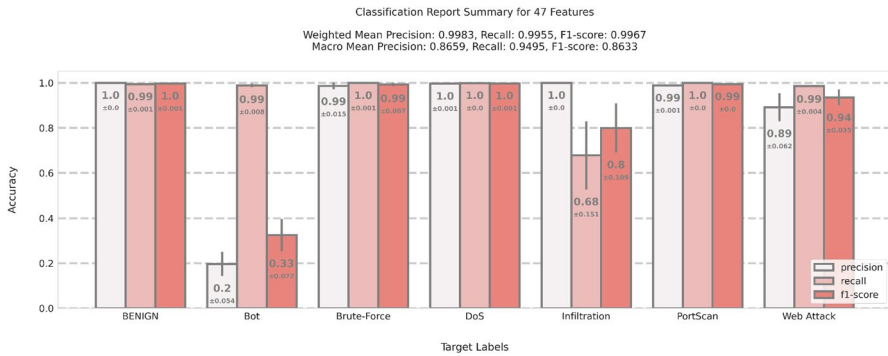


Fig. 12 Attack types metrics for 47 features

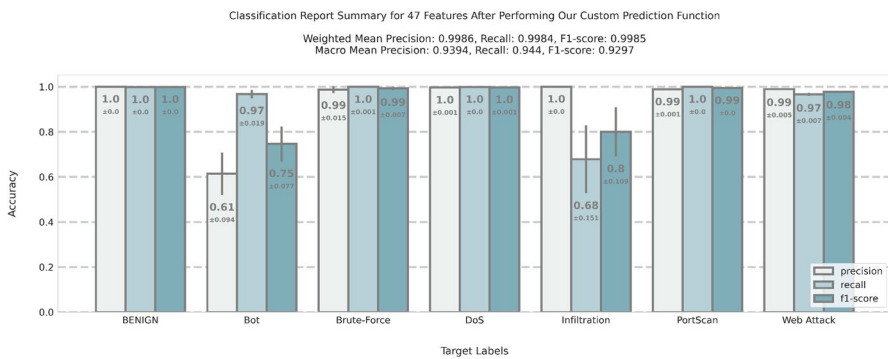


Fig. 13 Attack types metrics for 47 features with the custom prediction function

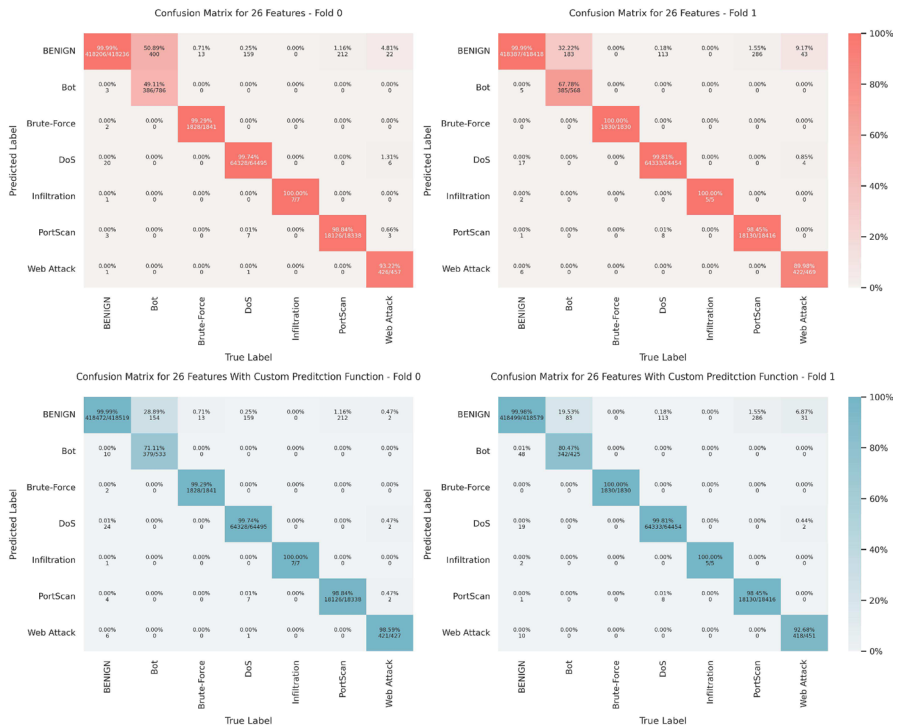


Fig. 14 Confusion matrix of 26 features with and without the custom prediction function (pred normalization)

recall and F1-scores for minority classes, particularly Bot and Web Attack. The 26 features likely included critical discriminative information, making this set the most effective for distinguishing between various attack types and benign traffic. Overall, the custom prediction function consistently improved model performance across all feature sets, particularly enhancing recall and F1-scores for minority attack classes. Its effectiveness in fine-tuning the model for diverse attack detection was demonstrated, along with its ability to address class imbalances and improve the model's generalization across attack categories.

The proposed model outperforms all models presented in Table 6 due to the custom prediction function. Due to the limitations of the Accuracy and Precision metrics, they are not useful in imbalanced datasets especially in Network Intrusion Detection Systems. In papers like [38], an F1-score of 99.8% is observed. However, the proposed model still outperforms it since the regular f1-score evaluates a single class compared to either the weighted f1-score, which takes into account the proportion of samples in each class when calculating the overall score, or the macro f1-score, which treats all classes equally, without regard to the number of samples in each class.

Due to the paper being in russian, the study's information was not able to be inserted into the table [34]. However, the authors achieved a 99.88% accuracy with the Random Forest classifier on the CICIDS2017 dataset, surpassing the 99.8%

Table 6 Comparison of Algorithms and Scores between Related Works and Proposed Model

Reference	Algorithm	Score type	Score
[39]	ANN, RF, 1-NN, EM, SOM	Accuracy	99.31% (ANN), 99.54% (RF), 99.49% (1-NN), 60.06% (EM), 59.06% (SOM)
[40]	KNN, improved KNN, LOF	Accuracy, detection rate	90.5% (Accuracy), 92.74% (Detection Rate for Zero-Day Attacks)
[41]	Artificial neural networks, random forest	Accuracy	90.909%
[38]	Random forest, decision tree	F1 score	~ 99.8%
[42]	DNN, Random forest, decision trees	Precision, recall, F1 score	Precision: ~ 0.96, Recall: ~ 0.97, F1 Score: ~ 0.96
[33]	Random forest, bayes net, random tree, naive bayes, J48	Accuracy	99.87% (J48), 99.86% (RF with 22 features)
Proposed model	Random forest, RandomizedSearchCV, Permutation importance	Weighted F1-score, Macro F1-score	0.9984 (Weighted F1), 0.9331 (Macro F1)

accuracy obtained in this research. Nevertheless, when comparing the two studies, it is important to consider variations in preprocessing methods, feature selection strategies, and the types of attacks examined. Although [34] may show better accuracy overall, the methods used in the proposed model prioritizes balancing class weights and targeting specific attack types to offer a strong solution with comparable performance. Furthermore, this research gives importance to reducing false positives and enhancing detection rates for specific complex, low-frequency attacks that might not be highlighted in [34].

The Random Forest algorithm excels in NIDS due to its ability to handle imbalanced data, rank feature importance, and resist overfitting, making it particularly effective for detecting network intrusions. However, it can be computationally complex, struggle with high-dimensional data, and be less interpretable compared to simpler models. While it generally outperforms other algorithms like Naive Bayes or SVM in accuracy, its performance can be context-dependent, particularly in scenarios requiring faster detection or greater model interpretability. Table 6 depicts the findings of this work that were contrasted with those of other studies that employed comparable or different methods for machine learning-based NIDS. The comparison showed that the suggested approach's accuracy and detection rate were competitive with other modern models. When put to the test on the CICIDS-2017 dataset, the suggested model did particularly well in terms of accuracy and detection rate. The comparison also showed that the suggested model's performance was based on the particular dataset and the kinds of threats being identified. On specific sorts of assaults or datasets, some models performed better than others, demonstrating how crucial it is to pick the right model for the situation.

Although this research shows encouraging outcomes from implementing a Random Forest-based NIDS with the CICIDS-2017 dataset, it is crucial to recognize specific constraints that could impact the overall applicability and strength of the conclusions. While the CICIDS-2017 dataset is extensive, it focuses on certain network behaviors and types of attacks that may not cover all the different threats seen in various real-world networks, which could restrict the model's usefulness in diverse settings. Moreover, the process of selecting features improves detection accuracy but may also increase the possibility of overfitting, causing the chosen features to be less effective in different datasets or real-time scenarios, resulting in a potential decrease in detection performance. The validation of the study was only done on the CICIDS-2017 dataset, raising doubts about the model's ability to adapt to other datasets with different network setups or threat environments. Moreover, the research's emphasis on a fixed dataset hinders evaluating the model's efficacy in dynamic, real-life situations with ongoing network traffic and changing threats, which can pose difficulties like managing live data streams and adjusting to new attack methods without regular retraining.

5 Conclusion

In conclusion, this study shows how feature importance and permutation impact can improve detection metrics through thorough experimentation. Methods like repeated stratified KFold, undersampling, and feature importance in addition to a custom prediction function were utilized in combination with a random forest classifier. The classifier used features 5, 15, 26, 36, and 47 to produce important results, with feature importance weights calculated by a permutation feature selection model employing a random forest. This method offered important understanding of how each feature impacts the macro and weighted f1-score of classification. The model attained a 99.8% weighted f1-score rate with 26 features for various attack types, although metric scores were lower for infiltration attacks because they occurred less frequently. The research emphasizes how feature selection affects model resilience, especially when dealing with minority-class intrusion attacks. Moreover, the real-world impacts of this study are significant. Enhancing the detection capabilities of NIDS helps organizations safeguard their networks against changing cyber threats, minimizing the chances of breaches and data loss.

The results also indicate that prioritizing feature selection can improve the effectiveness of intrusion detection systems, allowing them to be used in a wider range of network settings. In the future, the authors aim to investigate more feature significance methods and analyze in detail how different sets of features influence various types of assaults, with the goal of improving detection capabilities. Future studies will explore incorporating these results into NIDS implementations in real-time, possibly using AI and machine learning advancements to effectively respond to new threats and create a stronger cybersecurity system.

Moreover, Future research will look into feature combinations and their significance in more depth, with the help of permutation importance analysis. Moreover, we intend to investigate new techniques like SHAP (SHapley Additive exPlanations) by eXplainable AI, to gain understanding of how certain feature values impact model classifications. This will require analyzing the impact of different sets of features and their values on the detection of different types of attacks, thereby improving detection abilities. We will investigate integrating these discoveries into live NIDS setups, potentially using AI and machine learning advancements to promptly address new threats and enhance cybersecurity systems.

Funding Open access funding provided by The Science, Technology & Innovation Funding Authority (STDF) in cooperation with The Egyptian Knowledge Bank (EKB).

Data Availability The dataset analyzed during the current study is available on the website: <https://www.unb.ca/cic/datasets/ids-2017.html>.

The repository of the paper work can be found here: <https://www.kaggle.com/datasets/heshammousa/cicids-2017-models-results/>

Declarations

Conflict of interest The authors declare that they have no Conflict of interest to report regarding the present study.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

1. Ahmad, Z., Khan, A.S., Shiang, C.W., Abdullah, J., Ahmad, F.: Network intrusion detection system: a systematic study of machine learning and deep learning approaches. *Trans. Emerg. Telecommun. Technol.* (2020). <https://doi.org/10.1002/ett.4150>
2. Patel, S.K., Sonker, A.: Rule-based network intrusion detection system for port scanning with efficient port scan detection rules using snort. *Int. J. Future Gene. Commun. Netw.* **9**(6), 339–350 (2016). <https://doi.org/10.14257/ijfgen.2016.9.6.32>
3. Sathya, R., Abraham, A.: Comparison of supervised and unsupervised learning algorithms for pattern classification. *Int. J. Adv. Res. Artif. Intell.* (2013). <https://doi.org/10.14569/ijarai.2013.020206>
4. Maidamwar, P.R., Bartere, M.M., Lokulwar, P.P.: A survey on machine learning approaches for developing intrusion detection system. *SSRN Electron. J.* (2021). <https://doi.org/10.2139/ssrn.3833858>
5. Zhang, J., Li, H., Gao, Q., Wang, H., Luo, Y.: Detecting anomalies from big network traffic data using an adaptive detection approach. *Inf. Sci.* **318**, 91–110 (2014). <https://doi.org/10.1016/j.ins.2014.07.044>
6. Jyothsna, V., Prasad, V.V.R.: Fcaais: anomaly based network intrusion detection through feature correlation analysis and association impact scale. *ICT Express* **2**, 103–116 (2016)
7. Satoh, A., Ikenaga, T.: A flow-based detection method for stealthy dictionary attacks against secure shell. *J. Inf. Secur.* **21**, 31–41 (2015)
8. Juvonen, A., Hamalainen, T.: An efficient network log anomaly detection system using random projection dimensionality reduction. In: *Proc. 6th Int. Conf. New Technol., Mobility Secur. (NTMS)*, 1–5 (2014)
9. Chandrashekar, G., Sahin, F.: A survey on feature selection methods. *Comput. Electr. Eng.* **40**, 16–28 (2014)
10. Lou, H., Li, S., Zhao, Y.: Detecting community structure using label propagation with weighted coherent neighborhood propinquity. *Phys. A* **392**(14), 3095–3105 (2013). <https://doi.org/10.1016/j.physa.2013.03.014>
11. Ahmad, I., Hussain, M., Alghamdi, A., Alelaiwi, A.: Enhancing svm performance in intrusion detection using optimal feature subset selection based on genetic principal components. *Neural Computing and Applications* (2014). <https://doi.org/10.1007/s00521-013-1370-6>
12. Tama, B.A., Rhee, K.H.: A Combination of PSO-Based Feature Selection and Tree-Based Classifiers Ensemble for Intrusion Detection Systems. In: Park, D.S., Chao, H.C., Jeong, Y.S., Park, J. (eds.) *Advances in Computer Science and Ubiquitous Computing Lecture Notes in Electrical Engineering*. Springer, Singapore (2015). https://doi.org/10.1007/978-981-10-0281-6_71
13. Farnaaz, N., Akhil, J.: Random forest modeling for network intrusion detection system. *Proc. Comput. Sci.* **89**, 213–217 (2016). <https://doi.org/10.1016/j.procs.2016.06.047>
14. Pervez, M.S., Farid, D.: Feature selection and intrusion classification in nsl-kdd cup 99 dataset employing svms. *SKIMA 2014—8th International Conference on Software, Knowledge, Information Management and Applications*, (2015). <https://doi.org/10.1109/SKIMA.2014.7083539>
15. Chung, V., Wahid, N.: A hybrid network intrusion detection system using simplified swarm optimization (sso). *Appl. Soft Comput.* **12**, 3014–3022 (2012). <https://doi.org/10.1016/j.asoc.2012.04.020>
16. Barbara, D., Couto, J., Jajodia, S., Popyack, L., Wu, N.: Adam: detecting intrusions by data mining, 5–6 (2001)

17. Panda, M., Abraham, A., Das, S., Patra, M.: Network intrusion detection system: a machine learning approach. *Intell. Decis. Technol. J.* **5**, 347–356 (2011). <https://doi.org/10.3233/IDT-2011-0117>
18. Lee, W., Stolfo, S.: Data mining approaches for intrusion detection **7** (1998)
19. Lee, W., Stolfo, S.J.: A framework for constructing features and models for intrusion detection systems. *ACM Trans. Inf. Syst. Secur.* **3**(4), 227–261 (2000)
20. Debar, H., Becker, M., Siboni, D.: A neural network component for an intrusion detection system, 240–250 (1992). <https://doi.org/10.1109/RISP.1992.213257>
21. Mukkamala, S., Janoski, G., Sung, A.: Intrusion detection using neural networks and support vector machines. In: *Proceedings of the 2002 International Joint Conference on Neural Networks. IJCNN'02 (Cat. No.02CH37290)*, **2**, 1702–17072 (2002). <https://doi.org/10.1109/IJCNN.2002.1007774>
22. Panda, M., Patra, M.R.: Network intrusion detection using naïve bayes. (2007). <https://api.semanticscholar.org/CorpusID:6064057>
23. Amor, N., Benferhat, S., Elouedi, Z.: Naive bayes vs decision trees in intrusion detection systems **1**, 420–424 (2004). <https://doi.org/10.1145/967900.967989>
24. Lee, J.-H., Lee, J.-H., Sohn, S.-G., Ryu, J.-H., Chung, T.-M.: Effective value of decision tree with KDD 99 intrusion detection datasets for intrusion detection system. *IEEE* (2008)
25. Nguyen, H., Deokjai, C.: Application of data mining to network intrusion detection: classifier selection model, pp. 399–408 (1970). https://doi.org/10.1007/978-3-540-88623-5_41
26. Bhattacharya, S., Selvakumar, S.: Multi-measure multi-weight ranking approach for the identification of the network features for the detection of dos and probe attacks. *Comput. J.* **59**, 923–943 (2015). <https://doi.org/10.1093/comjnl/bxv078>
27. Chen, F., Ye, Z., Wang, C., Yan, L., Wang, R.: A feature selection approach for network intrusion detection based on tree-seed algorithm and k-nearest neighbor. In: *2018 IEEE 4th International Symposium on Wireless Systems Within the International Conferences on Intelligent Data Acquisition and Advanced Computing Systems (IDAACS-SWS)*, pp. 68–72 (2018). <https://doi.org/10.1109/IDAACS-SWS.2018.8525522>
28. Popoola, E., Adewumi, A.: Efficient feature selection technique for network intrusion detection system using discrete differential evolution and decision tree. *Int. J. Netw. Secur.* **19**, 660–669 (2017). [https://doi.org/10.6633/IJNS.201709.19\(5\).02](https://doi.org/10.6633/IJNS.201709.19(5).02)
29. Aljawarneh, S., Aldwairi, M., Bani Yassein, M.: Anomaly-based intrusion detection system through feature selection analysis and building hybrid efficient model. *J. Comput. Sci.* (2017). <https://doi.org/10.1016/j.jocs.2017.04.009>
30. el Boujnouni, M., Jedra, M.: New intrusion detection system based on support vector domain description with information gain metric. *Int. J. Netw. Secur.* **20**, 25–34 (2018)
31. Alhaj, T.A., Siraj, M.M., Zainal, A., Elshoush, H.T., Elhaj, F.: Feature selection using information gain for improved structural-based alert correlation. *PLoS ONE* **11**(11), 1–18 (2016). <https://doi.org/10.1371/journal.pone.0166017>
32. Hota, H., Shrivastava, A.: Decision tree techniques applied on nsl-kdd data and its comparison with various feature selection techniques. *Smart Innov. Syst. Technol.* **27**, 205–212 (2014). https://doi.org/10.1007/978-3-319-07353-8_24
33. Stiawan, D., Idris, M.Y.B., Bamhdi, A.M., Budiarto, R.: Cicans-2017 dataset feature analysis with information gain for anomaly detection. *IEEE Access* **8**, 132911–132921 (2020). <https://doi.org/10.1109/ACCESS.2020.3009843>
34. Goryunov, M.N., Matskevich, A.G., Rybolovlev, D.A.: Synthesis of a machine learning model for detecting computer attacks based on the cicans2017 dataset. *Proceedings of the Institute for System Programming of the RAS* (2020)
35. Chindove, H., Brown, D.: Adaptive machine learning based network intrusion detection. In: *Proceedings of the International Conference on Artificial Intelligence and Its Applications. icARTi '21. Association for Computing Machinery, New York, NY, USA* (2021). <https://doi.org/10.1145/3487923.3487938>
36. Bacevicius, M., Paulauskaite-Taraseviciene, A.: Machine learning algorithms for raw and unbalanced intrusion detection data in a multi-class classification problem. *Appl. Sci.* (2023). <https://doi.org/10.3390/app13127328>
37. Ko, K.-M., Baek, J.-M., Seo, B.-S., Lee, W.-B.: Comparative study of ai-enabled ddos detection technologies in sdn. *Appl. Sci.* (2023). <https://doi.org/10.3390/app13179488>

38. Reis, B., Maia, E., Praça, I.: Selection and performance analysis of cids2017 features importance. In: Benzekri, A., Barbeau, M., Gong, G., Laborde, R., Garcia-Alfaro, J. (eds.) *Foundations and Practice of Security*, pp. 56–71. Springer, Cham (2020)
39. Maseer, Z.K., Yusof, R., Bahaman, N., Mostafa, S.A., Foozy, C.F.M.: Benchmarking of machine learning for anomaly based intrusion detection systems in the cids2017 dataset. *IEEE Access* **9**, 22351–22370 (2021). <https://doi.org/10.1109/ACCESS.2021.3056614>
40. Elmasri, T., Samir, N., Mashaly, M., Atef, Y.: Evaluation of cids2017 with qualitative comparison of machine learning algorithm. In: 2020 IEEE Cloud Summit, 46–51 (2020). <https://doi.org/10.1109/IEEECloudSummit48914.2020.00013>
41. Pelletier, Z., Abualkibash, M.: Evaluating the cids-2017 dataset using machine learning methods and creating multiple predictive models in the statistical computing language r. *Science* **5**(2), 187–191 (2020)
42. Catillo, M., Del Vecchio, A., Pecchia, A., Umberto, V.: Transferability of machine learning models learned from public intrusion detection datasets: the cids2017 case study. *Softw. Qual. J.* (2022). <https://doi.org/10.1007/s11219-022-09587-0>
43. Almalag, A., Albadran, S., Mohamed, M.A.: An adaptive miner-misuse based online anomaly detection approach in the power system: an optimum reinforcement learning method. *Mathematics* **11**(4), 884 (2023). <https://doi.org/10.3390/math11040884>
44. Blum, A.: A temporal approach to unsupervised anomaly detection. Technical report, a temporal approach to unsupervised anomaly detection (2021). <https://doi.org/10.2172/1825324>
45. IDS 2017. <https://www.unb.ca/cic/datasets/ids-2017.html>. Accessed 23 July 2023
46. Jairu, P., Mailewa, A.B.: Network anomaly uncovering on cids-2017 dataset: A supervised artificial intelligence approach. In: 2022 IEEE International Conference on Electro Information Technology (eIT) (2022). <https://doi.org/10.1109/eit53891.2022.9814045>
47. Hnamte, V., Hussain, J.: Dependable intrusion detection system using deep convolutional neural network: a novel framework and performance evaluation approach. *Telemat. Inform. Rep.* **11**, 100070 (2023). <https://doi.org/10.1016/j.teler.2023.100077>
48. Henry, A., Gautam, S., Khanna, S., Rabie, K., Shongwe, T., Bhattacharya, P., Sharma, B., Chowdhury, S.: Composition of hybrid deep learning model and feature optimization for intrusion detection system. *Sensors* **23**(2), 890 (2023)
49. Panwar, S.S., Raiwani, Y.P., Panwar, L.S.: An intrusion detection model for cids-2017 dataset using machine learning algorithms. In: 2022 International Conference on Advances in Computing, Communication and Materials (ICACCM) (2022). <https://doi.org/10.1109/icaccm56405.2022.10009400>
50. Lin, H., Li, M.: Data pre-processing. In: *Practitioner's Guide to Data Science*, pp. 77–102. CRC Press (2023). <https://doi.org/10.1201/9781351132916-5>
51. Zelichenok, I.: Visualization and processing of information security events based on cids data 17. *Telecom IT* **9**(4), 49–55 (2021). <https://doi.org/10.31854/2307-1303-2021-9-4-49-55>
52. Sheikh, Y.: Effective feature selection for feature possessing group structure. *Int. J. Eng. Comput. Sci.* (2017). <https://doi.org/10.18535/ijecs/v6i5.19>
53. Prusty, S., Patnaik, S., Dash, S.K.: Skcv: stratified k-fold cross-validation on ml classifiers for predicting cervical cancer. *Front. Nanotechnol.* (2022). <https://doi.org/10.3389/fnano.2022.972421>
54. Kavya, R., Christopher, J., Panda, S.: Nearest neighbours and xai-based approach for soft labeling. In: *Proceedings of the 15th International Conference on Agents and Artificial Intelligence* (2023). <https://doi.org/10.5220/0011619800003393>
55. Sheikhi, S., Kostakos, P.: A novel anomaly-based intrusion detection model using psogwo-optimized bp neural network and ga-based feature selection. *Sensors* **22**(23), 9318 (2022). <https://doi.org/10.3390/s22239318>
56. Jiang, K., Wang, W., Wang, A., Wu, H.: Network intrusion detection combined hybrid sampling with deep hierarchical network. *IEEE Access* **8**, 32464–32476 (2020). <https://doi.org/10.1109/access.2020.2973730>
57. Bagui, S., Mink, D., Bagui, S., Subramaniam, S., Wallace, D.: Resampling imbalanced network intrusion datasets to identify rare attacks. *Future Int.* **15**(4), 130 (2023). <https://doi.org/10.3390/fi15040130>
58. Bagui, S., Li, K.: Resampling imbalanced data for network intrusion detection datasets. *J. Big Data* (2021). <https://doi.org/10.1186/s40537-020-00390-x>

59. Silva, B., Silveira, R., Silva Neto, M., Cortez, P., Gomes, D.: A comparative analysis of under-sampling techniques for network intrusion detection systems design. *J. Commun. Inform. Syst.* **36**(1), 31–43 (2021). <https://doi.org/10.14209/jcis.2021.3>
60. Megantara, A.A., Ahmad, T.: Feature importance ranking for increasing the performance of intrusion detection system. In: 2020 3rd International Conference on Computer and Informatics Engineering (IC2IE) (2020). <https://doi.org/10.1109/ic2ie50715.2020.9274570>
61. Reis, B., Maia, E., Praça, I.: Selection and performance analysis of CICIDS2017 features importance, pp. 56–71 (2020). https://doi.org/10.1007/978-3-030-45371-8_4
62. Disha, R.A., Waheed, S.: Performance analysis of machine learning models for intrusion detection system using gini impurity-based weighted random forest (giwrf) feature selection technique. *Cybersecurity* (2022). <https://doi.org/10.1186/s42400-021-00103-8>
63. Debeer, D., Strobl, C.: Conditional permutation importance revisited. *BMC Bioinform.* (2020). <https://doi.org/10.1186/s12859-020-03622-2>
64. Zakzouk, S., Ehab, M., Atef, S., Yousri, R., Tawfik, R.M., Darweesh, M.S.: A comparative study of machine learning techniques for automatic rice crop irrigation. In: 2021 17th International Computer Engineering Conference (ICENCO), pp. 46–50 (2021). IEEE
65. Opitz, J., Burst, S.: Macro F1 and macro F1. CoRR, Preprint at [arXiv:1911.03347](https://arxiv.org/abs/1911.03347) (2019)
66. Aurelio, Y.S., De Almeida, G.M., de Castro, C.L., Braga, A.P.: Learning from imbalanced data sets with weighted cross-entropy function. *Neural Process. Lett.* **50**, 1937–1949 (2019)

Publisher's Note Springer nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Mohamed Tarek Abdelaziz holds a BSc degree in Electronics and Computer Engineering from Nile University, Egypt (2024). His research interests focus on artificial intelligence, cyber security, and Operational Technology (OT) Security. He is passionate about leveraging AI to enhance cyber defense mechanisms and advancing security in industrial environments.

Abdelrahman Radwan holds a BSc in Computer Engineering from Nile University, Egypt (2024), with a GPA of 3.6. He is passionate about cybersecurity, particularly in web and network penetration testing, which he studied in the DEPI track. His expertise includes a 7-month Cyber Security Diploma, certifications like CCNA and MCSA, and his work on a machine learning-based Network Intrusion Detection System. He is eager to contribute to innovative teams in the cybersecurity field.

Hesham Mamdouh received his B.Sc. in Computer Engineering from Nile University, Egypt (2024), with the highest honors and a CGPA of 3.92/4.00. He is passionate about artificial intelligence, machine learning, IoT, and big data, with a focus on enhancing smart agriculture systems and network intrusion detection systems (NIDS). He has research experience in these areas, with this journal and a conference paper. Alongside academia, he has gained practical experience through internships and teaching assistant roles, working with machine learning and IoT applications in real-world situations.

Adel Saeed Saad holds a bachelor's degree in computer engineering from Nile University with a GPA of 3.58. He has an interest in embedded systems and machine learning, which he developed through a 6-month diploma program and an internship at a research center. His work experience includes a graduation project on a smart aquaponic system, where he applied IoT and machine learning together to improve the management of an aquaponic system. Adel is keen to apply his skills to computer engineering projects.

Abdulrahman Salem Abuzaid is a Communication and Electronics Engineering graduate from Helwan University, Cairo. He specializes in machine learning and artificial intelligence, with hands-on experience in deep learning and data science projects. His technical skills include Python, C++, TensorFlow, and PyTorch, alongside experience in NLP, and signal processing. He is passionate about AI and continuously expanding his expertise through practical projects.

Ahmed Ayman Abdelhakeem received his B.Sc. degree in Computer Engineering from Benha University, Egypt, in 2023. He is a Machine Learning Engineer with over 2 years of experience, specializing in classical machine learning, deep learning, and natural language processing. He has contributed to developing AI projects like an oral test system, focusing on improving performance and phoneme recognition accuracy. Ahmed also has experience as a Machine Learning Instructor, delivering training on topics such as supervised and unsupervised learning, model evaluation, and deployment. His research interests include speech recognition, AI for social good, and NLP applications.

Salma Zakzouk is a distinguished Computer Engineering graduate from Nile University, class of 2022, with a GPA of 3.96/4.00, earning the highest honors. Currently, she works as a Teaching Assistant at Nile University, specializing in AI, embedded systems, and IoT, with a strong emphasis on research. Salma has published seven research papers, five of which were completed during her undergraduate studies, including three from her A+ graded graduation project, "Lightweight Intelligent Food Recognition for Diabetes Management." She is now pursuing a Master's in Microelectronics Systems Design and has been recognized as the most impactful teaching assistant for 2022/23.

Kareem Moussa is an AI engineer who graduated from computer engineering, at Nile University, Egypt with the highest honors. He is currently a TA in electronics and computer engineering at Nile University. Kareem is pursuing his M.Sc. in AI engineering from the University of Science and Technology, Zewail City. He published 13 journal and conference research papers in AI, communications, and electronics. He has been a judge and reviewer in several competitions and conferences. Kareem has been a member of IEEE since 2021 (S'21, M'22).

M. Saeed Darweesh (M'17, SM'20) received his master's and Ph.D. degrees (with honors) in Electronics and Electrical Communications Engineering from the Faculty of Engineering, Cairo University, Giza, Egypt, in 2013 and 2017, respectively. He is currently an Associate Professor at the School of Engineering and Applied Sciences, at Nile University. Previously, he worked at the American University in Cairo (AUC), Zewail City (ZC) of Science and Technology, Arab Academy for Science and Technology and Maritime Transport (AASTMT), and Institute of Aviation Engineering and Technology (IAET). He is a Senior Member of the IEEE since 2020 and has served on major IEEE conferences' technical program committees. I hold several IEEE positions locally, regionally, and globally as the IEEE Egypt Section Secretary 2024-2025, IEEE SIGHT Egypt since June 2023, IEEE Consultants Network Vice Chair since April 2023, and the IEEE CAS Egypt Treasurer since January 2023. For the global and regional IEEE positions, starting January 2024, he is appointed as a corresponding member in the IEEE Region 8 Young Professionals Committee, a member in the IEEE HTB Proposal Evaluation Subcommittee, and a member in the IEEE HTB Events Committee. From May 2024, I have been appointed as Senior Editor at IEEE YP Impact Blog and a Vice Chair for the Education & Training subcommittee at IEEE PES Humanitarian Activities Committee (HAC). Also, I have been an IEEE HTB/SIGHT Projects Technical Reviewer since April 2023. Also, I was the IEEE Egypt Young Professionals Egypt Past Chair for 2022-2023. Saeed actively serves as a reviewer in several journals and conference publications, including IEEE conferences and journals. Saeed has led multiple research projects in the fields of artificial intelligence, machine learning, healthcare, and Industry 4.0. He has authored/coauthored over +90 papers in international journals and conferences. Also, he is a Principal Investigator (PI), Co-PI, and Research Associate in several research projects funded by national and international funding agencies. He has a solid technical background with a strong interest in machine learning and artificial intelligence. His research interests focus on Wireless Communications, Self-Driving Cars, Biomedical Engineering, Optimization Techniques, and Data Compression. Saeed also worked and trained with many telecom operators and suppliers like (MobiNil, Alcatel.Lucent, and Geniprocess) and has a strong background in computer networks and security.

Authors and Affiliations

**Mohammed Tarek Abdelaziz¹ · Abdelrahman Radwan¹ ·
Hesham Mamdouh¹ · Adel Saeed Saad¹ · Abdulrahman Salem Abuzaid³ ·
Ahmed Ayman AbdElhakeem⁴ · Salma Zakzouk^{1,2} · Kareem Moussa¹ ·
M. Saeed Darweesh^{1,2}**

✉ M. Saeed Darweesh
mdarweesh@nu.edu.eg

Mohammed Tarek Abdelaziz
moh.abdelaziz@nu.edu.eg

Abdelrahman Radwan
ab.radwan@nu.edu.eg

Hesham Mamdouh
hes.mamdouh@nu.edu.eg

Adel Saeed Saad
ad.saeed@nu.edu.eg

Abdulrahman Salem Abuzaid
abdulrahman.abdelglel.38@h-eng.helwan.edu.eg

Ahmed Ayman AbdElhakeem
ahmed180029@feng.bu.edu.eg

Salma Zakzouk
s.hossameldin@nu.edu.eg

Kareem Moussa
kamoussa@nu.edu.eg

¹ School of Engineering and Applied Sciences, Nile University, Giza 12677, Egypt

² Wireless Intelligent Networks Center (WINC), Nile University, Giza 12677, Egypt

³ Faculty of Engineering, Helwan University, Giza, Egypt

⁴ Faculty of Engineering, Benha University, Giza, Egypt